CASHFLOWIQ / MOBILE INTEGRATION

Flutter Apple Subscription API Integration Guide

StoreKit 2 purchase verification, premium status, restore, and history

DOCUMENT DETAILS

API version: v1

Backend path: /api/v1/subscription

Platform: Flutter / iOS / StoreKit 2

Updated: August 3, 2026

Audience: Flutter developers integrating authenticated Apple auto-renewable subscriptions.

1. Integration overview

The Flutter app completes a StoreKit 2 purchase and sends only the Apple transaction ID and expected product ID to the backend. The backend contacts Apple, verifies Apple-signed transaction data, stores the subscription, and returns premium access.

Security rule

Never calculate premium access from a client-provided purchase date, expiry date, boolean, receipt payload, or local device state. Treat the backend response as the source of truth.

Base URL

```text
https://YOUR_API_HOST/api/v1/subscription
```

Replace YOUR_API_HOST with the production or staging backend hostname. The legacy plural prefix /api/v1/subscriptions is also supported, but new Flutter code should use the singular path shown above.

Authentication

All four Flutter-facing subscription endpoints require an access JWT:

```http
Authorization: Bearer <accessToken>
Content-Type: application/json
```

The accessToken is returned by POST /api/v1/auth/login. A refresh token must not be used in this Authorization header.

Recommended app flow

- App starts or resumes: call GET /status and update premium UI.
- Purchase succeeds in StoreKit 2: call POST /verify.
- User selects Restore Purchases: call POST /restore.
- Subscription screen needs renewal records: call GET /history.
- On HTTP 401: refresh/login, then retry once.

2. Endpoint summary

POST	`/api/v1/subscription/verify`	Verify a completed StoreKit purchase.
GET	`/api/v1/subscription/status`	Get current backend-verified premium access.
POST	`/api/v1/subscription/restore`	Restore subscription ownership and history.

GET

`/api/v1/subscription/history`

Get verified Apple transaction history.

Backend-only routes

/api/v1/apple/webhook and /notifications/* are for Apple or administrators. The Flutter app must never call them.

3. Verify purchase

POST `/api/v1/subscription/verify`

Call immediately after StoreKit reports a verified purchase.

Request JSON

```json
{
  "transactionId": "100000123456789",
  "productId": "premium_monthly"
}
```

transactionId must be the StoreKit 2 transaction identifier. productId must exactly match the App Store Connect product identifier and the product allowlist configured on the backend.

Success: 200 OK

```json
{
  "success": true,
  "message": "Apple subscription verified successfully",
  "data": {
    "premium": true,
    "expiresAt": "2026-09-03T10:30:00.000Z"
  }
}
```

Flutter behavior

- Set premium UI from data.premium.
- Parse data.expiresAt as UTC using DateTime.parse(...).toUtc().
- Persist only for UI caching; re-check /status on the next app launch.
- Finish the StoreKit transaction after the backend verifies it successfully.

Do not send extra fields

The backend intentionally ignores client expiry, price, status, plan, environment, and purchase dates. Apple-signed values replace all client claims.

4. Subscription status

GET `/api/v1/subscription/status`

Call on launch, resume, login, and after purchase/restore.

Request body

No body. Send the Authorization header only.

Active success: 200 OK

JSON

```
{
  "success": true,
  "message": "Subscription status retrieved successfully",
  "data": {
    "premium": true,
    "expiresAt": "2026-09-03T10:30:00.000Z"
  }
}
```

No active subscription: 200 OK

JSON

```
{
  "success": true,
  "message": "Subscription status retrieved successfully",
  "data": {
    "premium": false,
    "expiresAt": null
  }
}
```

Billing grace period may return premium: true until Apple’s signed grace-period expiry. Billing retry without grace, expiry, refund, and revocation return premium: false.

5. Restore purchases

POST `/api/v1/subscription/restore`

Use from a user-initiated Restore Purchases action.

Request JSON

JSON

```
{
  "originalTransactionId": "100000123456789"
}
```

Use the original transaction ID from StoreKit 2. The backend fetches the complete Apple history, verifies each signed transaction, and associates the subscription with the authenticated CashFlowIQ user.

Success: 200 OK

JSON

```
{
  "success": true,
  "message": "Apple subscription restored successfully",
  "data": {
    "premium": true,
    "expiresAt": "2026-09-03T10:30:00.000Z"
  }
}
```

Account ownership protection

If an Apple original transaction is already linked to another CashFlowIQ account, the API returns HTTP 409. Do not silently switch accounts; explain that the subscription is linked elsewhere.

6. Subscription history

GET `/api/v1/subscription/history`

Returns verified purchase and renewal transactions.

Success: 200 OK

JSON


```
{
  "success": true,
  "message": "Apple subscription history retrieved successfully",
  "data": [
    {
      "purchaseDate": "2026-08-03T10:30:00.000Z",
      "expiresAt": "2026-09-03T10:30:00.000Z",
      "productId": "premium_monthly",
      "transactionId": "100000123456789",
      "originalTransactionId": "100000123456700",
      "environment": "Production",
      "revoked": false
    }
  ]
}
```


A user without subscription history receives data: [].

7. Error responses

All handled errors use this JSON structure:

```JSON
{
  "success": false,
  "message": "Human-readable error summary",
  "errorMessages": [
    {
      "path": "",
      "message": "Detailed error message"
    }
  ]
}
```

Important HTTP statuses

- 400 Bad Request — missing/invalid JSON, wrong product ID, or unsupported product.
- 401 Unauthorized — missing, malformed, expired, or invalid access token.
- 403 Forbidden — authenticated role cannot use the route.
- 404 Not Found — no Apple purchase history was found during restore.
- 409 Conflict — Apple subscription belongs to another app account.
- 502 Bad Gateway — Apple returned incomplete or inconsistent data.
- 503 Service Unavailable — Apple backend credentials/product mapping are unavailable.
- 500 Internal Server Error — unexpected backend failure; show retry UI and log safely.

Validation error example

```JSON
{
  "success": false,
  "message": "Validation Error",
  "errorMessages": [
    {
      "path": "transactionId",
      "message": "String must contain at least 1 character(s)"
    }
  ]
}
```

Recommended client policy

- Never retry 400 or 409 automatically.
- For 401, refresh authentication and retry once.
- Retry 429/500/502/503 with capped exponential backoff.
- Do not display raw server stack traces or Apple payloads to users.

8. Flutter HTTP example

Example using package:http. Adapt token storage and error classes to the app architecture.

DART

```dart
Future<Map<String, dynamic>> verifyApplePurchase({
  required String baseUrl,
  required String accessToken,
  required String transactionId,
  required String productId,
}) async {
  final response = await http.post(
    Uri.parse('$baseUrl/api/v1/subscription/verify'),
    headers: {
      'Authorization': 'Bearer $accessToken',
      'Content-Type': 'application/json',
    },
    body: jsonEncode({
      'transactionId': transactionId,
      'productId': productId,
    }),
  );

  final json = jsonDecode(response.body) as Map<String, dynamic>;
  if (response.statusCode != 200 || json['success'] != true) {
    throw ApiException(
      response.statusCode,
      json['message']?.toString() ?? 'Subscription verification failed',
    );
  }
  return json['data'] as Map<String, dynamic>;
}
```

Status model

DART

```dart
class PremiumStatus {
  final bool premium;
  final DateTime? expiresAt;

  PremiumStatus.fromJson(Map<String, dynamic> json)
      : premium = json['premium'] == true,
        expiresAt = json['expiresAt'] == null
            ? null
            : DateTime.parse(json['expiresAt'] as String).toUtc();
}
```

StoreKit package

Whichever Flutter in-app-purchase package is used, extract the StoreKit 2 transactionId/originalTransactionId from verified purchase data. Do not send Apple credentials, private keys, or server JWTs from Flutter.

9. Suggested Flutter state flow

On app launch or resume

- If authenticated, call GET /status.
- Show a neutral loading state while status is unknown.
- Apply premium entitlements only when data.premium is true.
- If offline, cached status may shape UI but must not authorize sensitive server features.

After StoreKit purchase

- Confirm the StoreKit result is locally verified by the StoreKit integration.
- Send transactionId and productId to POST /verify.
- When premium is true, finish/complete the StoreKit transaction.
- Refresh premium-gated screens and optionally fetch /history.

Restore action

- Start the platform restore/sync flow.
- Obtain originalTransactionId from the restored StoreKit transaction.
- Call POST /restore and use the returned premium status.

Logout

- Clear access/refresh tokens and cached subscription response.
- Do not carry premium state into a different CashFlowIQ login.

Testing

Use Apple Sandbox accounts and backend Sandbox auto-detection. Test new purchase, renewal, expiry, billing retry/grace, refund/revoke, restore, expired JWT, and subscription-linked-to-another-account behavior.

10. Flutter handoff checklist

- Production and staging API base URLs are configured per build flavor.
- Authorization uses the access token with the exact "Bearer " prefix.
- Product IDs exactly match App Store Connect and backend APPLE_PRODUCT_MAP.
- POST /verify runs after every successful StoreKit subscription purchase.
- GET /status runs on launch, resume, and after login.
- Restore Purchases is visible and calls POST /restore.
- All timestamps are parsed as UTC.
- Premium false and expiresAt null are handled normally.
- 401 refreshes authentication once; 409 shows an account-linking message.
- Flutter never calls Apple webhook or admin notification endpoints.
- No Apple private key, issuer ID, key ID, or server JWT is shipped in the app.
- StoreKit transaction is finished only after successful backend verification.

Backend contact information

Before release, obtain the final API hostname and exact production product IDs from the backend team. Those deployment-specific values are intentionally not embedded in this document.